

Compiler Construction Lab Manual

CS-471L - Fall 2025

University of Engineering and Technology Lahore

Department of Computer Science

Course Instructor: aliya.farooq@uet.edu.pk

Reference Book: Compilers: Principles, Techniques, and Tools (2nd Edition) - Aho, Lam, Sethi, Ullman (Dragon Book)

Course Learning Outcomes (CLOs)

CLO	Description	PLO	Assessment
CLO 1	Familiarity with Software Design and Programming at Large Issues	PLO0 1	MiscAssignments (20%)
CLO 2	Exposure to issues and implementation of lexical analysis	PLO0 3	LexicalAnalyzer (30%)
CLO 3	Exposure to issues and implementation of syntax analysis	PLO0 3	SyntaxAnalyzer (30%)
CLO 4	Use compiler construction tools like lex, YACC, bison	PLO0 5	AutomatedToolUsage (20%)

Week 1: Expression Evaluator

Learning Objectives

By the end of this lab, students will be able to:

- Understand the basic structure of an expression evaluator
- Implement tokenization of arithmetic expressions
- Build an expression parser using recursive descent
- Evaluate arithmetic expressions following operator precedence

Theoretical Background

Reference: Dragon Book Chapter 2 (A Simple Syntax-Directed Translator)

An expression evaluator is a simplified compiler that processes arithmetic expressions. It demonstrates the fundamental phases of compilation: lexical analysis (tokenization), syntax analysis (parsing), and semantic analysis (evaluation).

Expression Grammar

The grammar for simple arithmetic expressions follows operator precedence:

```
expr    → expr + term | expr - term | term
term    → term * factor | term / factor | factor
factor  → ( expr ) | number | identifier
```

To eliminate left recursion (required for recursive descent parsing):

```
expr    → term expr'
expr'   → + term expr' | - term expr' | ε
term    → factor term'
term'   → * factor term' | / factor term' | ε
factor  → ( expr ) | number | identifier
```

Lab Tasks

Task 1: Tokenizer Implementation

Create a tokenizer that breaks an expression string into tokens.

Token Types:

- NUMBER: Integer or floating-point numbers
- IDENTIFIER: Variable names (a-z, A-Z, followed by alphanumeric)
- PLUS, MINUS, MULTIPLY, DIVIDE: Arithmetic operators
- LPAREN, RPAREN: Parentheses
- EOF: End of input

Task 2: Recursive Descent Parser

Implement a recursive descent parser that:

1. Validates the expression syntax
2. Builds an Abstract Syntax Tree (AST)
3. Evaluates the expression

Task 3: Variable Support

Extend the evaluator to support:

- Variable assignment (e.g., `x = 5`)
- Variable lookup in expressions (e.g., `x + 3`)
- A symbol table to store variable values

Input Specification

Input Format: A text file or console input containing arithmetic expressions, one per line.

Sample Input:

```
3 + 4 * 2
(3 + 4) * 2
```

```
x = 10
y = 5
x + y * 2
((3 + 5) * 2) - 4 / 2
```

Output Specification

Output Format: For each expression, output the parsed tokens, the AST structure (optional), and the evaluated result.

Sample Output:

```
Expression: 3 + 4 * 2
Tokens: [NUMBER:3, PLUS, NUMBER:4, MULTIPLY, NUMBER:2]
Result: 11

Expression: (3 + 4) * 2
Tokens: [LPAREN, NUMBER:3, PLUS, NUMBER:4, RPAREN, MULTIPLY, NUMBER:2]
Result: 14

Expression: x = 10
Variable x assigned value 10

Expression: y = 5
Variable y assigned value 5

Expression: x + y * 2
Tokens: [IDENTIFIER:x, PLUS, IDENTIFIER:y, MULTIPLY, NUMBER:2]
Result: 20
```

Implementation Guidelines

Pseudocode for Expression Evaluation

```
function parseExpression():
    left = parseTerm()
    while currentToken is PLUS or MINUS:
        op = currentToken
        advance()
        right = parseTerm()
        left = BinaryNode(op, left, right)
    return left

function parseTerm():
    left = parseFactor()
    while currentToken is MULTIPLY or DIVIDE:
        op = currentToken
        advance()
        right = parseFactor()
        left = BinaryNode(op, left, right)
    return left

function parseFactor():
    if currentToken is NUMBER:
        value = currentToken.value
```

```
        advance()
        return NumberNode(value)
    else if currentToken is IDENTIFIER:
        name = currentToken.value
        advance()
        return VariableNode(name)
    else if currentToken is LPAREN:
        advance()
        node = parseExpression()
        expect(RPAREN)
        return node
```

Deliverables

1. Source code implementing the expression evaluator
2. Test cases demonstrating various expression types
3. Documentation explaining your implementation approach

Evaluation Criteria

Criteria	Marks
Correct tokenization	25%
Proper operator precedence	25%
Parentheses handling	20%
Variable support	20%
Code quality and documentation	10%